MICRO PROJECT

# Number System Converter using Python

| Detail | Entry |
|---|---|
| Name of Student | Sameer kumar<br>Aashutosh Kumar<br>Dibyanshu Kumar<br>Chandani Kumari<br>Ankit Kumar<br>Diwakar Kumar |
| Roll No. / Enrolment No. | 29-34 |
| Class / Semester | CSE / 3rd |
| Subject | Python Programming |
| | |
| Academic Year | 2026 - 2027 |

## 1. Title and Aim

Title: Number System Converter using Python

Aim: To design and develop a simple, menu-driven Python program that converts a number between the Decimal, Binary, Octal and Hexadecimal number systems, and that reports an error when the entered number is not valid for the chosen system.

## 2. Introduction of the Project

A number system is a way of representing numbers using a fixed set of digits. Every system has a base (or radix) that tells how many different digits it uses. People normally work with the decimal system, but computers store and process all data in binary. Octal and hexadecimal are shorter ways of writing binary numbers and are widely used in programming, memory addressing, colour codes and networking.

The four number systems handled by this project are summarised below.

| Number System | Base | Digits Used | Example |
|---|---|---|---|
| Decimal | 10 | 0 - 9 | 156 |
| Binary | 2 | 0, 1 | 10011100 |
| Octal | 8 | 0 - 7 | 234 |
| Hexadecimal | 16 | 0 - 9 and A - F | 9C |

The program uses decimal as a common middle step. The input number is first converted to decimal using the positional value of each digit, and the decimal value

is then converted to every other base by repeated division. For example, 2F in hexadecimal is 2 x 16 + 15 = 47 in decimal, which is 101111 in binary and 57 in octal. The program shows all four results at once, so the user does not have to run separate conversions.

## 3. Tools Used

### 3.1 Hardware

| Component | Specification |
|---|---|
| Computer / Laptop | Desktop PC or laptop |
| Processor | Intel Core i3 or equivalent (or higher) |
| RAM | 4 GB or more |
| Storage | At least 500 MB free disk space |
| Input / Output | Keyboard and monitor |

### 3.2 Software

| Software | Purpose |
|---|---|
| Windows 10 / 11 | Operating system |
| Python 3.8 or later | Programming language used to write and run the converter |
| IDLE / VS Code | Editor used to write and save the program (converter.py) |
| Command Prompt | Used to run the program and view the output |
| Microsoft Word | Used to prepare this project report |

## 4. Procedure

### 4.1 Steps followed

1. Setup: Install Python 3 and open IDLE or VS Code.
2. New file: Create a new file named converter.py.
3. Convert to decimal: Write the function to_decimal(), which reads each digit of the input, checks that it is valid for the chosen base, and builds the decimal value using value = value x base + digit.
4. Convert from decimal: Write the function from_decimal(), which divides the decimal value repeatedly by the target base and collects the remainders in reverse order.
5. Menu: Write the menu in main(). The user chooses the input system (1 - 4) or exits (5), then types the number.
6. Validation: Raise and catch a ValueError when a digit does not belong to the

chosen system (for example 2 in a binary number) and print a clear error message.

7. Display: Display the Decimal, Binary, Octal and Hexadecimal values of the number and show the menu again.
8. Run: Save the file and run it with the command python converter.py
9. Test: Test with valid and invalid inputs and note down the outputs.

## 4.2 Algorithm

To decimal: set value = 0; for each digit from left to right, check the digit is valid for the base, then set value = value x base + digit.

From decimal: while value > 0, take value mod base as the next digit (placed on the left) and set value = value div base; if the value is 0 the answer is 0.

## 4.3 Program code (converter.py)

```
"""Number System Converter - Micro Project
Converts numbers between Decimal, Binary, Octal and Hexadecimal.
"""

DIGITS = "0123456789ABCDEF"
BASES = {"1": ("Decimal", 10), "2": ("Binary", 2), "3": ("Octal", 8), "4": ("Hexadecimal", 16)}


def to_decimal(number: str, base: int) -> int:
    """Convert a string in the given base to a decimal integer."""
    number = number.strip().upper()
    if not number:
        raise ValueError("Input is empty.")
    value = 0
    for ch in number:
        if ch not in DIGITS[:base]:
            raise ValueError(f"'{ch}' is not a valid digit for base {base}.")
        value = value * base + DIGITS.index(ch)
    return value


def from_decimal(value: int, base: int) -> str:
    """Convert a decimal integer to a string in the given base."""
    if value == 0:
        return "0"
    result = ""
    while value > 0:
        result = DIGITS[value % base] + result
        value //= base
    return result


def convert(number: str, src: int, dst: int) -> str:
    return from_decimal(to_decimal(number, src), dst)


def show_menu():
    print("=" * 44)
    print("       NUMBER SYSTEM CONVERTER")
    print("=" * 44)
```

```
    for key, (name, base) in BASES.items():
        print(f"  {key}. {name} (base {base})")
    print("  5. Exit")
    print("-" * 44)


def main():
    while True:
        show_menu()
        choice = input("Select INPUT number system (1-5): ").strip()
        if choice == "5":
            print("Thank you for using the converter. Goodbye!")
            break
        if choice not in BASES:
            print("Invalid choice. Please enter 1-5.\n")
            continue
        src_name, src_base = BASES[choice]
        number = input(f"Enter the {src_name} number: ")
        try:
            decimal_value = to_decimal(number, src_base)
        except ValueError as err:
            print(f"Error: {err}\n")
            continue
        print(f"\nResults for {src_name} {number.strip().upper()}:")
        for key, (name, base) in BASES.items():
            print(f"  {name:<12}: {from_decimal(decimal_value, base)}")
        print()


if __name__ == "__main__":
    main()
```

## 5. Output Screenshots

The screenshots below show the program running in the Command Prompt. The text in yellow is what the user types. The results were produced by running the program above.

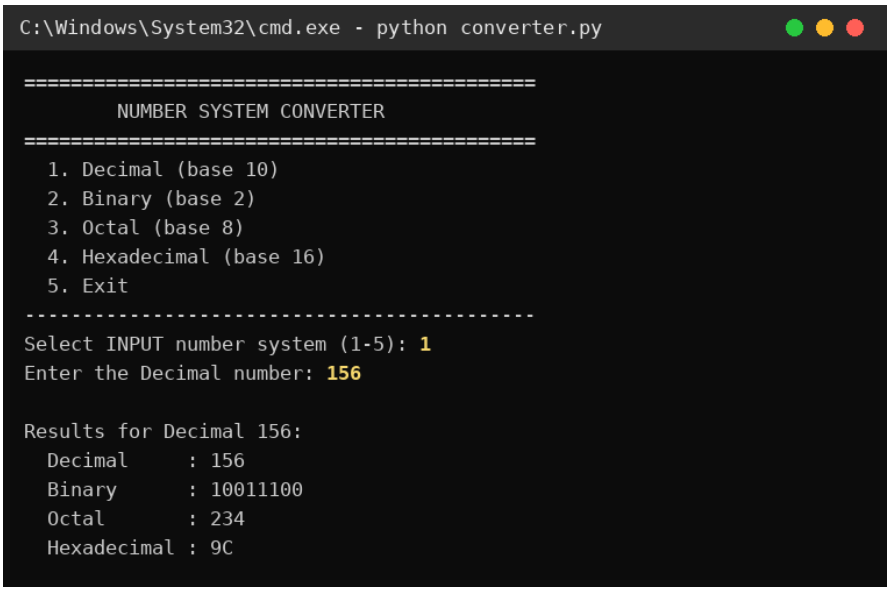



*Figure 1: Decimal 156 converted to all four systems*

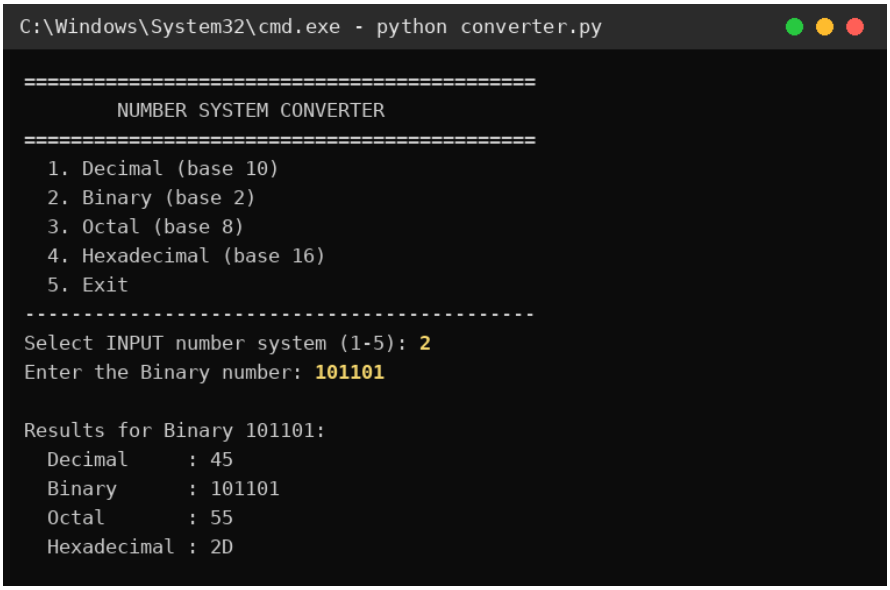



*Figure 2: Binary 101101 converted to all four systems*

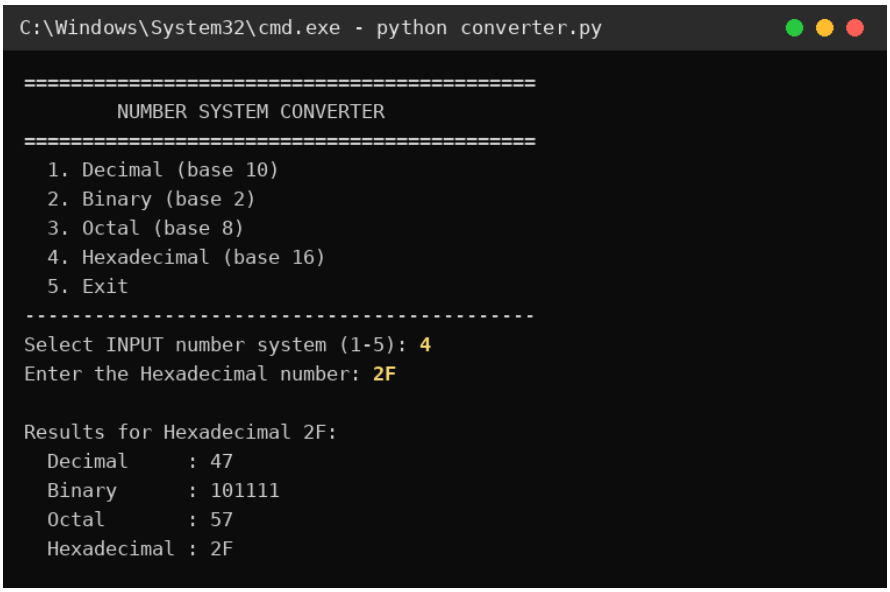



*Figure 3: Hexadecimal 2F converted to all four systems*

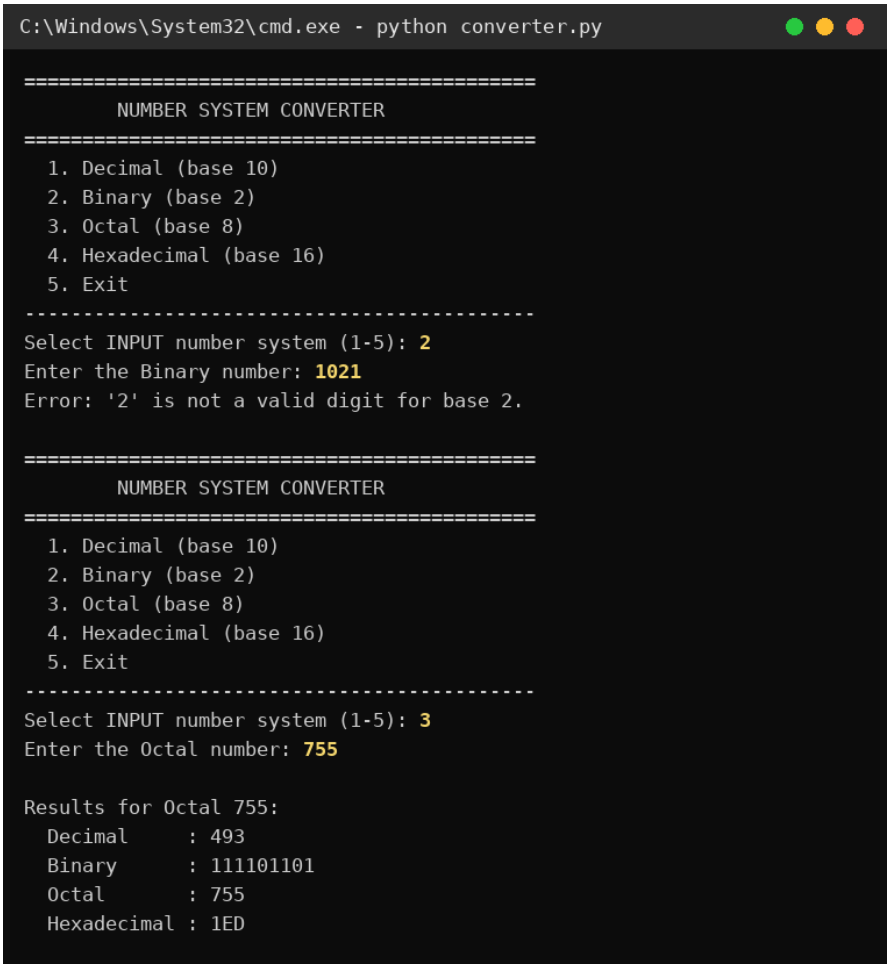



*Figure 4: Error for invalid binary 1021 followed by Octal 755*

## Summary of test cases

| Input System | Input | Decimal | Binary | Octal | Hex |
|---|---|---|---|---|---|
| Decimal | 156 | 156 | 10011100 | 234 | 9C |
| Binary | 101101 | 45 | 101101 | 55 | 2D |
| Hexadecimal | 2F | 47 | 101111 | 57 | 2F |
| Octal | 755 | 493 | 111101101 | 755 | 1ED |
| Binary | 1021 | Error: '2' is not a valid digit for base 2 | | | |

# 6. Conclusion

The microproject successfully converts numbers between the decimal, binary, octal and hexadecimal systems using Python. The program takes the number in any one system and displays its value in all four systems at once. Invalid digits are detected and reported with a clear message, and the menu lets the user make as many conversions as needed in one run.

While building the project, the concepts of number systems, positional values, loops,

functions, string handling and exception handling were practised. The results were checked against manual calculations and were found correct.

## 7. Future Scope

- Fractions and negatives: Support fractional numbers such as 10.625 and negative numbers (two's complement).
- Graphical interface: Build a Tkinter window with input boxes and buttons instead of the text menu.
- Custom bases: Allow any base from 2 to 36, not only the four common ones.
- Step-by-step working: Show the division and remainder steps so students can learn how each conversion works.
- Batch conversion: Convert a whole file or list of numbers at once and save the results to a text or CSV file.
- Web or mobile version: Convert the program into a web page or mobile app so it can be used on any device.